

Documentație Tehnică de Proiect

Sistem de Gestiune și Rezervare pentru Cinematografe (CinemaCMS)

Student: **Volentiru Răzvan**

Grupa: **3121b**

Facultate: **Inginerie Electrică și Știința Calculatoarelor**

Mediu de Lucru: **Linux Ubuntu (WSL) & Ncurses**

1. Introducere și Obiective practice

Acest proiect reprezintă o aplicație software capabilă să ruleze direct în terminalul Linux, depășind limitările clasice de tip text simplu „linie cu linie”. Scopul principal este realizarea unei interfețe grafice interactive în consolă (TUI - Text User Interface), optimizată pentru ecrane tactile sau interacțiune hibridă mouse-tastatură, destinată operatorilor sau casierilor dintr-un multiplex cinematografic.

Designul elimină complet formularele rigide de introducere a datelor prin text. Utilizatorul poate naviga fluid cu săgețile, poate căuta în timp real un film prin simpla tastare a numelui sau poate selecta dintr-o singură privire scaunele dintr-o sală folosind click-ul de mouse. Această abordare pune accent pe experiența utilizatorului (UX) și pe performanța de randare în medii cu resurse hardware limitate.

2. Structurile de Date: Cum reprezentăm cinematograful în memorie

Pentru a modela o rețea reală de cinematografe, am organizat codul folosind concepte OOP clare și structuri imbricate (agregare). Fiecare element are o responsabilitate unică în memorie.

```
struct Seat {  
    int r, c;  
    bool res;  
    string owner;  
    bool selected = false;  
};
```

Explicație cod: Aceasta este unitatea de bază (scaunul). Reține coordonatele pe matrice (rând și coloană), starea de ocupare (res - adevărat dacă este vândut) și numele clientului. Câmpul selected este secretul din spatele selecției multiple: el funcționează ca un coș de cumpărături temporar, reținând dacă utilizatorul a dat click pe scaun fără să fi confirmat încă plata.

```
struct Screening {
    string movie, date, time;
    double price;
    vector<vector<Seat>> seats;
};
```

Explicație cod: Structura reprezintă o rulare (proiecție) concretă a unui film. Pe lângă metadata esențiale precum titlul, data calendaristică, ora și prețul unui bilet, ea găzduiește o matrice bidimensională dinamică (vector<vector<Seat>>). Această matrice reprezintă fizic sala de proiecție alocată acelui film, permițând ca fiecare film să aibă stări de rezervare complet independente.

3. Salvarea Datelor: Persistența în format JSON

Pentru ca datele și rezervările să nu se piardă la închiderea programului, am implementat un modul de persistență care scrie automat stările pe disc în format JSON. Am preferat maparea manuală în detrimentul macro-urilor generice pentru a asigura portabilitatea codului pe orice versiune de compilator GCC din Linux.

```
void to_json(json& j, const Screening& s) {
    j = json{{"movie", s.movie}, {"date", s.date}, {"time", s.time}, {"price", s.price}, {"seats", s.seats}};
}
void from_json(const json& j, Screening& s) {
    j.at("movie").get_to(s.movie);
    j.at("date").get_to(s.date);
    j.at("time").get_to(s.time);
    j.at("price").get_to(s.price);
    j.at("seats").get_to(s.seats);
}
```

Explicație cod: Aceste funcții sunt apelate nativ de librăria `nlohmann/json`. Atunci când scriem `json(network)`, structura ierarhică parcurge automat aceste funcții recursiv, transformând obiectele din memorie în text structurat. La pornirea aplicației, funcția inversă preia textul din fișier și reconstruiește instantaneu întreaga ierarhie de cinematografe, săli și scaune ocupate.

4. Motorul Grafic TUI și Controlul Hibrid (Mouse & Tastatură)

Interfața vizuală solidă se bazează pe biblioteca `ncurses`. Am configurat aplicația să ruleze în modul **Immediate Mode GUI**, ceea ce înseamnă că ecranul se redesenează exclusiv atunci când utilizatorul generează o acțiune (apasă o tastă sau dă click). Acest lucru reduce utilizarea procesorului la aproape 0% în perioadele de inactivitate.

4.1 Controlul Mouse-ului și gestionarea click-urilor

```
if (ch == KEY_MOUSE) {
    MEVENT ev;
    if (getmouse(&ev) == OK && (ev.bstate & BUTTON1_CLICKED)) {
        if (checkClick(ev, btnBack)) { ... }
        for (int r = 0; r < max_r; r++) {
            int dy = offY + r * 2;
            if (ev.y == dy && ev.x >= dx && ev.x <= dx + 3) { ... }
        }
    }
}
```

Explicație cod: În momentul în care ncurses detectează un semnal de la mouse, structura MEVENT preia coordonatele exacte ale ecranului (coloana X și rândul Y). Codul rulează un algoritm de coliziune geometrică: compară poziția cursorului de mouse cu poziția unde au fost desenate butoanele de control sau scaunele. Dacă se constată o potrivire, starea elementului se modifică instantaneu.

5. Module Core: Multi-Select, Live Search și Raportare

5.1 Căutarea Instantanee în Timp Real

Pentru a gestiona volume mari de proiecții generate automat la prima pornire, aplicația implementează un filtru live. În ecranele de selecție, utilizatorul nu trebuie să apese pe un câmp text dedicat; poate începe să tasteze direct pe tastatură.

```
for(int i=0; i<items.size(); i++) {
    if(searchQ.empty() || toLowerStr(items[i]).find(toLowerStr(searchQ)) != string::npos)
    {
        filtered.push_back(i);
    }
}
```

Explicație cod: La fiecare tastă apăsată, buffer-ul searchQ stochează șirul introdus. Aplicația convertește temporar atât textul din baza de date, cât și căutarea în litere mici (case-insensitive) prin funcția utilitară toLowerStr și filtrează vectorul pe ecran. Utilizatorul vede rezultatele actualizându-se instantaneu sub degetele sale.

5.2 Logica de Selecție Multiplă și Calculul de Preț

Sistemul permite selectarea mai multor locuri în avans. În momentul în care casierul finalizează comanda și apasă butonul de checkout, se calculează totalul de plată.

```

int selectedCount = 0;
for(auto& r : scr.seats)
    for(auto& c : r)
        if(c.selected) selectedCount++;

char buf[50];
snprintf(buf, sizeof(buf), "TOTAL: %.2f RON (%d bilete)", selectedCount * scr.price,
selectedCount);
string client = inputDialog("Numele clientului:", string(buf));

```

Explicație cod: Aplicația scanează matricea pentru a număra câte scaune au indicatorul `selected` setat pe adevărat. Multiplică valoarea obținută cu prețul specific al biletului alocat filmului curent și afișează valoarea formatată în interiorul ferestrei de dialog. La introducerea numelui, locurile trec din starea galbenă (selectat) în cea roșie (rezervat definitiv).

6. Ghid Practic de Compilare și Execuție

Pentru a asigura o randare stabilă și corectă în terminalul nativ Ubuntu WSL, proiectul trebuie asamblat folosind flag-ul de optimizare `-O3`, care instruește compilatorul GCC să optimizeze buclele matriceale de desenare.

Comanda de compilare optimizată:

```
g++ -O3 -std=c++17 main.cpp -o cinemax -lncurses
```

Comanda de pornire:

```
./cinemax
```

Cont de Test	Parolă Criptată (Hash)	Nivel de Permisuni în Sistem
admin	admin	Drepturi CRUD complete. Poate adăuga/șterge cinematografe, săli, proiecții și are permisiunea de a reseta complet o sală vândută.
casier	casier	Drepturi limitate de vânzare (Operator). Poate doar să caute proiecții, să verifice disponibilitatea locurilor și să emită bilete noi.